CS4001NI Programming

50% Individual Coursework

2025-26 Autumn

Credit: 30

Student Name: Rijan Bhattarai

London Met ID: 25029641

College ID: NP01AI4A250028

Assignment Due Date: Monday, March 9, 2026

Assignment Submission Date: Monday, March 9, 2026

Module Leader: Mr. Bikram Poudel

Word Count:

Contents

1. Introduction 1
2. Class Diagram 1
3.Method Description 2
   3.1 AI Model 2
   3.2 Personal Plan 4
   3.3 Pro Plan 6
4.Testing 8
   Test 1 – Valid Prompt 8
   Test 2 – Context Limit Exceeded 8
   Test 3 – Monthly Limit Exhausted and Purchase Prompts 9
   Test 4 – Pro Plan Infinite Prompts 10
5. Error Detection and Correction 11
   Error 1 — Syntax Error: Missing Semicolon 11
   Error 2 — Logical Error 11
   Error 3 — Not a Statement Error 12
6. Conclusion 13
Bibliography 14
Code Listings 15
   AIMode.java 15
   Personal Plan 16
   Pro Plan 17
   Main 18

Table of Figures

Figure 1: Class Diagram 1
Figure 2: Test 1 8
Figure 3: Test 2 9
Figure 4: Test 3 9
Figure 5: Test 4 10
Figure 6: Error 1 11
Figure 7: Error 2 11
Figure 8: Error 3 12
Figure 9: AI Model.java 15
Figure 10: PersonalPlan.java 16
Figure 11: ProPlan.java 17
Figure 12: Main.java 18

1. Introduction

Introducing the project, a subscription-based, a subscription business model is one in which customers are charged a recurring fee for access to a product instead of a one-time expense (SBIGrowth, 2026), AI models for teams or individual customers. The project is differentiated into **Personal Plan** and **Pro Plan**, where personal plans are directed towards individual customers and pro plan directed towards teams due to features of team management.

The goal of this project is to track monthly quotas, enrollment of new prompts or adding team members, the project also clearly differentiates between pro and personal plans and set the restrictions for each accordingly, verify entered data for both classes.

The programming language used to develop the software is Java. Java is a high-level, object-oriented programming language developed by Sun Microsystems in 1995 (GeeksForGeeks, 2019). The reason java is used for this project is due to its scalable nature, secure platform, frameworks for enterprise development and open-source libraries (GeeksForGeeks, 2019).

2. Class Diagram

“A class diagram illustrates the structure of a system by showing its classes, attributes, methods, and the relationships among them” (Oracle Corporation, 2024). Its primary purpose is to give the developers the architecture of the program using graphics and images (Alhadi Klaib, 2021). It is convenient for both experienced and inexperienced developers who can clearly implement the overarching base architecture of the program using graphics.

The diagram below shows the IS-A inheritance relationship between the abstract parent class AI Model and its two concrete subclasses, Personal Plan and Pro Plan. Each class box displays the class name, instance variable names with data types, and the full signatures of all constructors and methods. (Oracle Corporation, 2024)

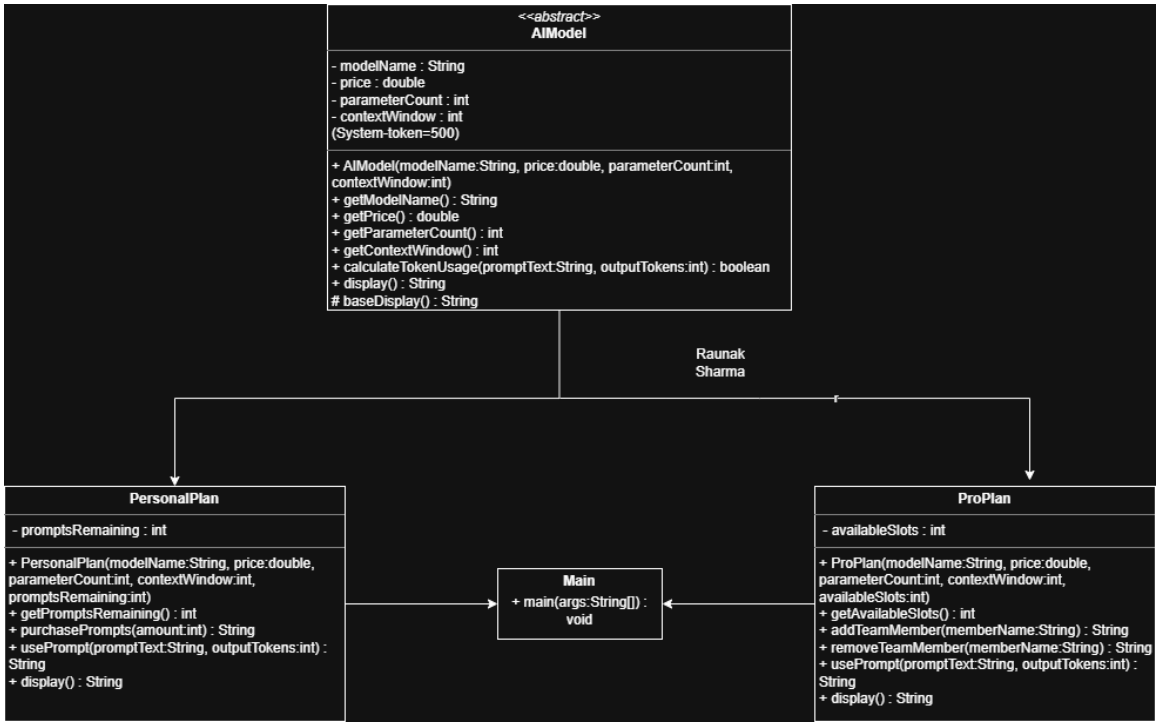

Figure 1: Class Diagram

3.Method Description

This section provides a detailed description of every method defined in AIModel.java, PersonalPlan.java, and ProPlan.java, including the constructor for each class.

3.1 AI Model

Table 1: AIModel.java method descriptions.

Method Name	**AI Model**
Parameter(s)	String model Name, double price, long parameter, String context Size
Return Type	N/A
Description	Constructor for the model
Internal Working	Initializes each object for the program to be used in further calculation.

Method Name	**Get Model Name**
Parameter(s)	None
Return Type	String
Description	A simple accessor method for getting model name.
Internal Working	Returns the model's name

Method Name	**Get Price**
Parameter(s)	None
Return Type	double
Description	Accessor methods for returning the price per 1 lakh tokens in Nepali Rupees (NPR).
Internal Working	Returns the price of the model in NPR

Method Name	**Get Parameter Count**
Parameter(s)	None
Return Type	long
Description	An accessor function for the attribute
Internal Working	Returns parameters

Method Name	**Get Context Size**
Parameter(s)	None

Return Type	Int
Description	An accessor method for the attribute context size
Internal Working	Returns the context window size of a model

Method Name	**Base Display**
Parameter(s)	None
Return Type	String
Description	Returns the string for the baseline for future outputs
Internal Working	Uses **string format** to insert the value of objects into the base display for future display outputs.

Method Name	**Display – Abstract**
Parameter(s)	None
Return Type	String
Description	An abstract function that needs to be implemented by child class which inherit the AI Model class.
Internal Working	**Base Display** + **Display** is used to show the proper output for the user.

Method Name	**Calculate Token Usage**
Parameter(s)	Prompt, Expected Amount
Return Type	Boolean
Description	Returns a Boolean value depending whether the total token count is less than **context window size**
Internal Working	Estimates the input token using GPT formula (OpenAI, 2023), calculates the total token usage and returns true or false depending whether the total token usage is less or equal to the context window size.

3.2 Personal Plan

Table 2: PersonalPlan.java method descriptions.

Method Name	**Personal Plan**
Parameter(s)	prompts, model Name, price, parameter, context Size
Return Type	N/A
Description	Initializes a personal plan object with a monthly prompt quota and all four AI model attributes.
Internal Working	Initializes the objects for usage in the program

Method Name	**Get Prompts**
Parameter(s)	None
Return Type	int
Description	Accessor Method for accessing the number prompts
Internal Working	Returns number of prompts remaining the monthly quota

Method Name	**Purchase Prompts**
Parameter(s)	Add Prompts
Return Type	String
Description	Allows user to purchase prompts to add more prompts in their monthly quota
Internal Working	Adds prompt when the number of purchasing prompts is a positive value greater than zero and rejects the values that are less than one.

Method Name	**Use Prompt**
Parameter(s)	Prompt, Expected Output Token
Return Type	String
Description	A method that sends the prompt to the AI model
Internal Working	Checks if the user has exhausted their monthly quota or not if they have exhausted send assert an error else send the prompt to the AI model for the user to interact with the model.

Method Name	Display - **Abstract**
Parameter(s)	None
Return Type	String

Description	Displays the total info for the user to interact with the program
Internal Working	Concatenates the baseline display that is inherited from AI model class and adds extra information from the personal plan class for the user to get all the information about the model they have selected.

3.3 Pro Plan

Table 3: ProPlan.java method descriptions.

Method Name	**Pro Plan**
Parameter(s)	Team Slots, model Name, price, long parameter, String context Size
Return Type	N/A
Description	Constructor for the class Pro Plan Class.
Internal Working	Inherits from the parent class AI Model Class and initializes its own objects

Method Name	**Get Available Slots**
Parameter(s)	None
Return Type	int
Description	A simple accessor method for returning the number of members in the plan
Internal Working	Returns the number of slots remaining in the team

Method Name	**Add Team Member**
Parameter(s)	Member Name
Return Type	String
Description	A method to add team members in the current plan
Internal Working	Adds a team member to the plan if slots are available and decrements team slots by one on success

Method Name	**Remove Team Member**
Parameter(s)	Member Name
Return Type	String
Description	Removes a team member from the team
Internal Working	Removes a named team member from the plan and frees up a slot. Returns an error message if the member is not found in the list.

Method Name	**Use Prompt**
Parameter(s)	Prompt, Expected Output Token
Return Type	String
Description	A method that sends the prompt to the AI model
Internal Working	Checks if the user has exhausted their monthly quota or not if they have exhausted send assert an error else send the prompt to the AI model for the user to interact with the model.

Method Name	Display - **Abstract**
Parameter(s)	None
Return Type	String
Description	Displays the total info for the user to interact with the program
Internal Working	Concatenates the baseline display that is inherited from AI model class and adds extra information from the pro plan class for the user to get all the information about the model they have selected.

4.Testing

This process is also defined as the software testing phase. Software testing is an important process in the Software Development Lifecycle (SDLC**)**. It involves verifying and **validating** that a Software Application is free of bugs, meets the technical requirements set by its Design and Development, and satisfies user requirements efficiently and effectively. (GeeksForGeeks, 2026)

Test 1 – Valid Prompt

A prompt is the specific text input or command given to an AI model to generate a particular, desired output. (Vocabulary, 2026). The given test is conducted on both

```
(Test 1)
Personal Plan
Prompt accepted!
Prompt: Lebonbon DA GOAT
Prompts remaining: 1

(Test 1)
Pro Plan
Prompt accepted!
Prompt: Lakers Winning the 2026 Championship
```

Figure 2: Test 1

Test 2 – Context Limit Exceeded

A "Context limit exceeded" error occurs when the total number of tokens (words, characters, and code) in your prompt, including conversation history and retrieved data, surpasses the maximum capacity of the AI model (Rabadon Skyler, 2026)

```
(Test 2)
Personal Plan
Error: Context Limit Exceeded.
Context Window: 1000

(Test 2)
Pro Plan
Error: Context limit exceeded.
Context Limit: 5000
Try shortening your prompt or reducing the output length.
```

Figure 3: Test 2

Test 3 – Monthly Limit Exhausted and Purchase Prompts

When the user exhausts the monthly quota for their plan, they get prompted with an error message saying their quota has been exhausted they have the option to purchase new prompts or upgrade to a pro plan.

```
(Test 3)
Personal Plan
Prompt accepted!
Prompt: Assignment Finished?
Prompts remaining: 0

(Test 3.1)
Personal Plan
Error: Monthly quota exhausted. Please purchase more prompts or upgrade to Pro Plan.

(Test 3.2)
Personal Plan
Purchase successful!
Prompts added: 5
Prompts remaining: 5
```

Figure 4: Test 3

Test 4 – Pro Plan Infinite Prompts

This test sets to demonstrate that the pro plan that teams have have infinite number of prompts.

```
(Test 4)
Pro Plan
Prompt accepted!
Prompt: Hi

Pro Plan
Prompt accepted!
Prompt: Generate a pic

Pro Plan
Prompt accepted!
Prompt: Rijan is so amazing wow
```

Figure 5: Test 4

5. Error Detection and Correction

This section documents three distinct errors encountered during implementation — one syntax error, one runtime error, and one logic error — along with the steps taken to identify and resolve each one.

Error 1 — Syntax Error: Missing Semicolon

Error Type	Syntax
Description	A semicolon was omitted at the end of a variable assignment inside the Personal Plan constructor
Identification	BlueJ highlighted the line in red. The compiler output read: 'error: ';' expected', with the caret pointing directly at the missing symbol.
Solution	Added the missing semicolon. Re-compiled successfully with no further errors.

Evidence:

```
super(modelName, price, parameterCount, contextWindowSize);
this.promptsRemaining = promptsRemaining
```

Figure 6: Error 1

Error 2 — Logical Error

Error Type	Runtime
Description	'!' was missing on the function calculateTokenUsage where it is checking for tokens used
Identification	It was identified after running the program when every output was inverted
Solution	The solution was to add the '!' operator infront of calculateTokenUsage(prompt, expectedAmount)

Evidence:

```
if (calculateTokenUsage(prompt, expectedAmount))
{
    return String.format(
            "Error: Context limit exceeded.%n" +
            "Context Limit: %d%n" +
            "Try shortening your prompt or reducing the output length.",
            getContextWindowSize());
}
```

Figure 7: Error 2

Error 3 — Not a Statement Error

Error Type	Logic
Description	In calculate token usage the instead of the assignment operator the equal to operator was used
Identification	It was identified when the program didn't run when it was set to run, and blue J highlighted in red saying it was not a statement.
Solution	Removing the extra '=' and making the equal to operator an assignment operator.

Evidence:

```
final int SYSTEM_TOKENS == 100;
```

Figure 8: Error 3

6. Conclusion

This assignment provided a practical opportunity to apply core Object-Oriented Programming principles in Java. By designing and implementing the AIModel class hierarchy, comprising the abstract parent class AIModel and its two concrete subclasses PersonalPlan and ProPlan, a deeper understanding of inheritance, encapsulation, polymorphism, and method overriding was developed.

PersonalPlan demonstrated how a child class can extend parent functionality by managing a prompt quota, enforcing business rules (rejecting invalid purchases), and overriding getDetails() to include plan-specific information. Similarly, ProPlan illustrated the use of ArrayList for dynamic data management, showing how team membership can be tracked and modified at runtime.

Debugging the three errors in Section 5 reinforced the importance of careful data validation. The Logical Error highlighted the necessity of checking the conditions properly while writing the code to not run into more run time error that occur due to logic.

Overall, this milestone has strengthened practical Java skills, improved understanding of class design, and introduced systematic testing strategies all of which will be built upon in Milestone 2 with the addition of the GUI SubscriptionGUI class.

Bibliography

Alhadi Klaib, 2021. Integrating Class Diagrams. *ResearchGate,* 1(1), p. 10.

Anon., 2025. *Programming language.* [Online]
Available at: https://www.computerhope.com/jargon/p/programming-language.htm

GeeksForGeeks, 2019. *geeksforgeeks,org.* [Online]
Available at: http://geeksforgeeks.org/java/introduction-to-java/
[Accessed 8 3 2026].

GeeksForGeeks, 2026. *GeeksForGeeks.* [Online]
Available at: https://www.geeksforgeeks.org/software-testing/software-testing-basics/
[Accessed 3 3 2026].

J., B., 2018. *Effective Java 3rd edn.* Boston: Addison-Wesley: s.n.

OpenAI, 2023. *What is input tokens and how to calculate them,* San Francisco : OpenAI.

Oracle Corporation, 2024. *Java SE 21 Documentatation,* San Fransico: Oracle Cororation.

Oracle Corporation, 2024. *Oracle SE 21 Development.* [Online].

Rabadon Skyler, 2026. *Portkey.* [Online]
Available at: https://portkey.ai/error-library/context-length-error-10013
[Accessed 8 3 2026].

SBIGrowth, 2026. Subscription Business Model. *Wikiepdia,* 2(III), p. 5.

Vocabulary, 2026. *Vocabulary.* [Online]
Available at: https://www.vocabulary.com/dictionary/prompt
[Accessed 8 3 2026].

Code Listings

AIMode.java

```java
public abstract class AIModel {
    private String modelName;
    private double price;
    private int parameterCount;
    private int contextWindowSize;

    public AIModel(String modelName, double price, int parameterCount, int contextWindowSize) {
        this.modelName = modelName;
        this.price = price;
        this.parameterCount = parameterCount;
        this.contextWindowSize = contextWindowSize;
    }

    // Accessor Methods
    public String getModelName() {
        return this.modelName;
    }
    public double getPrice() {
        return this.price;
    }
    public int getParameterCount() {
        return this.parameterCount;
    }
    public int getContextWindowSize() {
        return this.contextWindowSize;
    }

    // Display Functions
    public abstract String display();

    public String baseDisplay(){
        return String.format(
                "Model Name: %s" +
                "%nPrice: Rs %.2f" +
                "%nParameter Count: %d Billion" +
                "%nContext Window Size: %d",
                modelName, price, parameterCount, contextWindowSize
                );
    }

    // Token Usage
    public boolean calculateTokenUsage(String prompt, int expectedAmount) {
        /*
         * Input Tokens Reference: https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them
         * */

        final int SYSTEM_TOKENS = 100;

        int inputTokens = prompt.trim().length() / 4;
        int total = inputTokens + expectedAmount + SYSTEM_TOKENS;

        return total <= contextWindowSize;
    }
}
```

Figure 9: AI Model.java

Personal Plan

```java
public class PersonalPlan extends AIModel {
    private int promptsRemaining;
    private static final String plan = "Personal Plan";

    // Constructor
    public PersonalPlan(int promptsRemaining,
                        String modelName,
                        double price,
                        int parameterCount,
                        int contextWindowSize) {

        super(modelName, price, parameterCount, contextWindowSize);
        this.promptsRemaining = promptsRemaining;
    }

    // Accessor method
    public int getPromptsRemaining() {
        return promptsRemaining;
    }

    // Purchase Prompt
    public String purchasePrompts(int amount) {
        if (amount <= 0) {
            return "Error: Purchase failed. Please enter a number greater than 0 or Upgrade to Pro Plan";
        }
        promptsRemaining += amount;
        return String.format(
                "Purchase successful!" +
                "%nPrompts added: %d" +
                "%nPrompts remaining: %d",
                amount, promptsRemaining);
    }

    // Accept User Prompt
    public String usePrompt(String prompt, int expectedAmount) {
        // Checks
        if (promptsRemaining < 1) {
            return "Error: Monthly quota exhausted. Please purchase more prompts or upgrade to Pro Plan.";
        }


        if (!calculateTokenUsage(prompt, expectedAmount)) {
            return String.format("Error: Context Limit Exceeded.%n" +
                    "Context Window: %d", getContextWindowSize());
        }

        // both checks passed
        promptsRemaining--;

        return String.format(
                "Prompt accepted!%n" +
                "Prompt: %s" +
                "%nPrompts remaining: %d",
                prompt, promptsRemaining);
    }

    // Display Method
    @Override
    public String display() {
        return  baseDisplay() + String.format(
                "%nPlan Type: %s%n" +
                "Prompts Remaining: %d",
                plan, promptsRemaining);
```

Figure 10: PersonalPlan.java

Pro Plan

```java
public class ProPlan extends AIModel {
    private int availableSlots;
    private static final String plan = "Pro Plan";
    public ProPlan(int availableSlots,
                   String modelName,
                   double price,
                   int parameterCount,
                   int contextWindowSize) {

        super(modelName, price, parameterCount, contextWindowSize);

        this.availableSlots = availableSlots;
    }
    // Accessor method
    public int getAvailableSlots() {
        return availableSlots;
    }

    public String addTeamMember(String memberName) {
        if (availableSlots <= 0) {
            return "Error: No slots available.";
        }
        availableSlots--;
        return String.format
                (
                "Team member added!%n" +
                "Member name: %s%n" +
                "Slots remaining: %d",
                memberName, availableSlots
                );
    }
    // Remove Member Method
    public String removeTeamMember(String memberName)
    {
        availableSlots++;
        return String.format(
                "Team member removed!%n" +
                "Member name: %s%n" +
                "Slots remaining: %d",
                memberName, availableSlots);
    }
    public String usePrompt(String prompt, int expectedAmount)
    {

        // Perform Checks
        if (!calculateTokenUsage(prompt, expectedAmount))
        {
            return String.format(
                    "Error: Context limit exceeded.%n" +
                    "Context Limit: %d%n" +
                    "Try shortening your prompt or reducing the output length.",
                    getContextWindowSize());
        }
        return String.format(
                "Prompt accepted! %n" +
                "Prompt: %s%n",
                prompt);
    }
    // Display Method
    @Override
    public String display()
    {
        return  baseDisplay() + String.format("%nPlan Type: %s%n" +"Available Slots: %d", plan, availableSlots);
    }
```

Figure 11: ProPlan.java

Main

```java
public class Main {

    public static void main(String[] args) {
        PersonalPlan freeTier = new PersonalPlan(2, "ATLAS", 223.2322, 1000, 1000);
        ProPlan proTier = new ProPlan(2, "GOTHAM", 1000.0000, 10000, 5000);

        // Test 1
        String resultTest_1_free = freeTier.usePrompt("Lebonbon DA GOAT", 2000);
        System.out.println("(Test 1)\nPersonal Plan\n"+ resultTest_1_free);
        System.out.println();
        String resultTest_1_pro = proTier.usePrompt("Lakers Winning the 2026 Championship", 50000);
        System.out.println("(Test 1)\nPro Plan\n"+ resultTest_1_pro);

        // Test 2
        System.out.println();
        String resultTest_2_free = freeTier.usePrompt("McLaren Winning F1 2026 Season", 100);
        System.out.println("(Test 2)\nPersonal Plan\n"+ resultTest_2_free);
        System.out.println();

        String resultTest_2_pro = proTier.usePrompt("McLaren Winning F1 2026 Season", 500);
        System.out.println("(Test 2)\nPro Plan\n"+ resultTest_2_pro);
        System.out.println();

        // Test 3
        String resultTest_3_free = freeTier.usePrompt("Assignment Finished?", 2);
        String resultTest_31_free = freeTier.usePrompt("Hmm, REDRUM REDUMM RAHHHHHH", 1);
        System.out.println("(Test 3)\nPersonal Plan\n"+ resultTest_3_free);
        System.out.println();
        System.out.println("(Test 3.1)\nPersonal Plan\n"+ resultTest_31_free);
        System.out.println();

        String resultTest_32_free = freeTier.purchasePrompts(5);
        System.out.println("(Test 3.2)\nPersonal Plan\n"+ resultTest_32_free);
        System.out.println();

        // Test 4
        String resultTest_4_pro = proTier.usePrompt("Hi", 2);
        String resultTest_41_pro = proTier.usePrompt("Generate a pic", 2);
        String resultTest_42_pro = proTier.usePrompt("Rijan is so amazing wow", 2);

        System.out.println();
        System.out.println("(Test 4)\nPro Plan\n"+ resultTest_4_pro);
        System.out.println("Pro Plan \n"+ resultTest_41_pro);
        System.out.println("Pro Plan \n"+ resultTest_42_pro);

    }
}
```

Figure 12: Main.java